# ONE-SITE & YARDI REPORTING PORTAL

## Technical Scope, Mac Execution Rationale, and Delivery Plan

| | |
|---|---|
| **Prepared for** | Marc Menowitz, CEO — ApartmentCorp |
| **Prepared by** | Brandon Rose, Special Projects |
| **Date** | September 1, 2026 |

**Executive conclusion:** The OneSite & Yardi Reporting Portal is a production internal management platform, not a prototype. It is usable from any authorized computer or phone, while the Mac remains the appropriate provider-execution workstation today because the existing OneSite runner was built and validated for macOS and a visible Microsoft Edge session. The portal's conservative outside-development value is **$57,000–$81,750** at $75/hour. The Windows work computer remains a viable future path only after its separate secure bridge passes policy, installation, and readiness gates.

*Prepared for internal leadership review*

**Prepared for:** Marc Menowitz, CEO — ApartmentCorp

**Prepared by:** Brandon Rose, Special Projects

**Date:** September 1, 2026

**Purpose:** This document explains what the OneSite & Yardi Reporting Portal entails, why it is a multi-week engineering program rather than a one-week task, and why the existing Mac is the appropriate provider-execution environment for the current OneSite workflow.

---

## Executive conclusion

The OneSite & Yardi Reporting Portal is a production internal management platform, not a prototype. It allows authorized management users to request, generate, file, review, and distribute operational reports through one responsive portal usable from any authorized Windows PC, Mac, iPhone, or Google Fold device. The portal itself is universally accessible; the only device-specific component is the carefully controlled provider-execution workstation required to interact with OneSite and, later, Yardi.

For OneSite reporting today, the Mac is the lowest-risk execution path because the existing runner was built, registered, and tested for macOS and a visible Microsoft Edge session. The company-managed Windows work computer remains a viable future option, but it requires a separately reviewed Windows Edge bridge, policy validation, installation, and readiness testing before it can safely replace the current Mac workflow.

---

## 1. What the product is

The OneSite & Yardi Reporting Portal, deployed as the **OneSite Reporting Hub**, is a production internal management platform that lets authorized management users request, generate, file, review, and distribute operational reports from two separate property-management systems: **RealPage OneSite** and **Yardi**. It is designed for use from any computer or phone, including iPhone and Google Fold devices.

The critical architectural fact — and the reason this project is measured in weeks rather than days — is that **neither RealPage OneSite nor Yardi offers a public API for report automation** [1] [2]. Both vendors expose data through interactive, authenticated web sessions protected by CAPTCHA and two-factor verification. Everything the portal does must therefore be built around a secure, auditable bridge between a normal web application and a human-signed-in browser session, without ever touching, storing, or transmitting provider credentials.

## 2. The eight major subsystems

The portal is not one application; it is eight coordinated subsystems, each of which is a meaningful software project on its own.

| # | Subsystem | What it does |
|---|---|---|
| 1 | **Responsive web portal** | The management-facing application: individual user accounts, source-specific access control, audit visibility, request creation, status tracking, and a full report library. Works on desktop and mobile. |
| 2 | **Live report catalog synchronization** | The OneSite Classic Reports catalog — **310 reports across 10 paginated pages** — is read from a live Edge session and kept in sync so the portal's report selector reflects the provider catalog. A separate, fully isolated Yardi catalog flow is in staged development. |
| 3 | **Property directory synchronization** | The authoritative property list is read from OneSite's Classic-mode property dropdown and synchronized into the portal's property directory, covering the full 42-property portfolio. |
| 4 | **Request workflow engine** | Users select a report, set report-specific parameters, and run it for **all properties or one selected property**. All-property runs must use OneSite's explicit *Select All* behavior — not a visible-page subset — requiring dedicated preflight and inspection logic. |
| 5 | **Self-hosted runner infrastructure** | Provider-side work is executed by a private GitHub Actions self-hosted runner (`BrandonRose2/realpage-portal-runner`) registered to a designated management computer. The runner verifies a live authenticated Edge session, processes approved requests, and downloads completed outputs. |
| 6 | **Document filing and storage pipeline** | Every output is filed under separate source roots — `OneSite Reporting/{Property}/…` and `Yardi Reporting/{Property}/…` — preserving original Excel/PDF files, request metadata, status, and generated HTML summaries, with duplicate protection and idempotent filing. |
| 7 | **Report library and viewers** | Filed reports are filterable and sortable by source, date, property, and report type. PDFs open in an in-portal viewer; Excel workbooks receive a sheet-aware preview and original-file download; every workbook can receive a mobile-readable responsive HTML companion. |
| 8 | **Manager workflow and communications** | The portal generates property-specific, editable **Manager's Checklists** with per-line verification, auto-save, progress bars, “Needs attention” deep links, and a “Submit for Review” flow; it matches manager contacts and drafts, but |

| # | Subsystem | What it does |
|---|---|---|
| | | never auto-sends, review emails. |

## 3. The security model — and why it is non-negotiable

Because OneSite and Yardi sessions are protected by CAPTCHA and two-factor authentication, the portal is built around a hard security boundary:

> **The portal, the runner, the repository, the logs, and the chat never collect, store, relay, or expose OneSite or Yardi usernames, passwords, browser cookies, session tokens, or MFA codes. Provider login and CAPTCHA/2FA completion remain human actions inside a provider-approved, visible Microsoft Edge session on a designated management computer.**

This requires a credential-free runner configuration, a narrowly scoped temporary Microsoft Edge companion restricted to the My Reports download context for a single approved batch, complete **provider separation** between OneSite and Yardi, and fail-closed reconciliation logic that refuses to act unless request name, format, property counts, and filing pairs all match.

## 4. The current engineering problem: Request #60001 and the Windows transition

The exact recovery point demonstrates why this is real engineering rather than button-clicking:

- **Request #60001** — the "All Units (Excel)" portfolio run dated August 28, 2026 — used OneSite's Select All across **35 selected property options**, producing results across multiple paginated My Reports pages. Ten original workbooks and ten HTML companions were filed before the run froze.
- The post-run downloader repeatedly scanned only the **first ten visible rows** of the paginated My Reports grid and failed to wait for page transitions, so the remaining completed outputs were not reliably captured.
- A later Edge companion was restricted to a **21-property filing plan**, which conflicts with the recorded 35-option selection. That discrepancy **cannot safely be resolved by rerunning the report** — rerunning would create duplicate provider-side results and destroy the audit trail. It must be reconciled by building a single truth table across provider My Reports entries, portal document records, and staged files, then filing only genuinely missing, already-completed original/HTML pairs.
- The registered execution machine is a macOS self-hosted runner using macOS-specific AppleScript tab control, while the current day-to-day workstation is a **company-managed Windows PC**. Since the existing automation layer is macOS-specific, a separate, security-reviewed Windows Edge bridge has been designed. It includes an Edge extension, a local native-messaging host pinned to one exact extension ID, a versioned nonce-protected message protocol, and staged validation from environment preflight through read-only inventory and non-duplicative filing.

All of this work — the recovery audit, queue cleanup, Windows preflight branch, loopback bridge, installer, and 15+ automated security tests — has been completed and validated **without a single provider action**: no report rerun, no email, no provider-setting change, and no credential exposure.

## 5. Why the Mac is the better execution environment for this reporting workflow — at this stage

The point is **not** that a Mac is universally better than a Windows computer for all ApartmentCorp work. The web portal itself is intentionally device-independent. The distinction is limited to the highly controlled **provider-side execution environment** for RealPage OneSite and, later, Yardi.

For the current reporting workflow, the Mac is the safer and more efficient choice because it is the environment for which the operational runner was actually built, registered, and tested. The existing self-hosted runner is a macOS/ARM64 runner with the required `self-hosted`, `macOS`, and `onesite-runner` labels; its OneSite integration uses macOS-specific AppleScript controls to communicate only with an **already open, visible Microsoft Edge tab** in Brandon's signed-in desktop session. It does not launch a separate browser profile, store credentials, or attempt to bypass OneSite's CAPTCHA or two-factor authentication.

| Decision factor | Existing Mac execution path | Windows work-computer path today |
|---|---|---|
| **Runner compatibility** | The existing runner and GitHub Actions workflow were built and labeled for macOS. | The current production workflow will not route to Windows; it requires a separately labeled Windows workflow. |
| **Visible Edge control** | The current implementation uses a tested macOS-native AppleScript control layer for the user's visible Edge tab. | Windows requires a new Edge extension plus native-messaging bridge because macOS AppleScript cannot run on Windows. |
| **Security and credential boundary** | Uses the already authenticated visible Edge session while keeping credentials, cookies, tokens, and MFA information outside the portal and repository. | Can achieve the same boundary only after the new bridge, extension identity pinning, policy checks, installer, and readiness tests are completed and reviewed. |
| **Operational readiness** | The remaining issue is repair of the existing GitHub runner host (`Runner.Listener` is missing), not a redesign of provider interaction. | The Windows approach is in staged readiness design. Its loopback package is deliberately not yet a production OneSite integration and cannot reconcile Request #60001. |
| **Company-device policy** | The Mac path uses the already-defined management-machine boundary. | The Windows machine is company-managed, so its native host/extension must satisfy enterprise policy or obtain narrowly scoped administrator approval. |
| **Risk and time to resume** | Repairing the runner preserves the previously tested code path and requires the fewest moving parts. | Moving immediately to Windows adds implementation, packaging, security-review, installation, and validation work before any report work can resume. |

**Bottom line:** Continuing with the Mac for OneSite reporting is not a preference for one brand of computer. It is the lowest-risk route because it preserves a tested, credential-free, provider-safe execution path. Using Windows before the dedicated Windows bridge is fully built and validated would mean replacing the automation layer in the middle of a live-report reconciliation — exactly the type of avoidable change that creates reporting errors, duplicate files, and security risk.

The correct operating model is therefore straightforward: managers can create, review, and access reports from **any device** through the portal, while the Mac temporarily remains the designated **provider-execution workstation** until the existing runner is repaired and, if desired, the separate Windows bridge passes its staged security and readiness gates.

## 6. Why "less than a week" was never a realistic expectation

The table below breaks the portal into the work packages an outside development shop would quote at the requested **$75/hour** blended rate. These are conservative midpoints for a senior full-stack developer; agencies typically quote $125–$200/hr for comparable integration work [3] [4].

| Work package | Est. hours | Cost @ $75/hr |
|---|---|---|
| Responsive multi-user web portal (accounts, access control, audit, mobile) | 160–220 | $12,000–$16,500 |
| Live OneSite catalog sync (310 reports, 10 pages) + property directory sync | 60–90 | $4,500–$6,750 |
| Request workflow engine (parameters, Select-All preflight, queue) | 80–120 | $6,000–$9,000 |
| Self-hosted GitHub Actions runner + live Edge session control layer | 100–140 | $7,500–$10,500 |
| Download, filing, and idempotent reconciliation pipeline | 70–100 | $5,250–$7,500 |
| Report library, PDF viewer, Excel preview, HTML companion generation | 60–90 | $4,500–$6,750 |
| Manager's Checklist system (editable, auto-save, submit-for-review) | 60–80 | $4,500–$6,000 |
| Email drafting workflow + contacts-directory matching | 30–50 | $2,250–$3,750 |
| Windows Edge bridge (extension + native host + installer + tests) | 80–120 | $6,000–$9,000 |
| Security architecture, credential isolation, staged validation, and documentation | 60–80 | $4,500–$6,000 |
| **Total** | **760–1,090 hrs** | **$57,000–$81,750** |

In calendar terms, that is roughly **4.5 to 6.5 months of one full-time senior developer** — or a $60,000–$80,000 fixed-bid engagement from a consulting shop before ongoing maintenance. The expectation that this be finished in less than a week is not anchored to the actual scope of the system.

## 7. Recommended next steps

The immediate low-risk path is to repair the existing Mac runner, reconcile Request #60001 without rerunning it, and then begin the isolated Yardi catalog flow. A Windows runner remains a valid future option, but it is a separate security-reviewed build rather than a one-click substitution. This sequence ensures that nothing touches live provider data until every safeguard is verified, protecting the company's OneSite and Yardi accounts, resident PII, and financial reporting integrity.

---

## References

[1]: RealPage — OneSite property management platform (https://www.realpage.com/)

[2]: Yardi — property management software (https://www.yardi.com/)

[3]: U.S. Bureau of Labor Statistics — Software Developers, Occupational Outlook Handbook (https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm)

[4]: Upwork — How much does it cost to hire a software developer (https://www.upwork.com/hire/software-developers/cost/)